ORDONNANCEMENT

Prenez le temps de lire attentivement ce document. Il contient notamment certaines instructions que vous devez absolument respecter. Tout manquement influencera votre note.
D'autres éléments d'informations (clarification, complément) pourront vous être fournis ultérieurement.

PROGRAMME A DEVELOPPER

Votre programme se déroule en plusieurs étapes :

1. Lecture d'un tableau de contraintes contenu dans un fichier .txt, mise en mémoire et affichage de ce tableau sur l'écran ;
2. Construction d'un graphe correspondant à ce tableau de contraintes ;
3. Vérification que ce graphe ne possède pas de circuit et qu'il n'y a pas d'arcs à valeur négative.
4. Si ces propriétés sont vérifiées, calculer le calendrier au plus tôt, la durée totale du projet, le calendrier au plus tard, les marges et le(s) chemin(s) critique(s).

   Pour le calcul du calendrier au plus tard, utilisez la convention que la date au plus tard de fin de projet soit égale à sa date au plus tôt. Comme vous savez, pour le calcul des calendriers il faut d'abord effectuer le tri topologique du graphe : ordonner les sommets dans l'ordre des rangs croissants. Il faut donc affecter un rang à chaque sommet, en utilisant un algorithme de votre choix parmi ceux que vous avez vu en cours et/ou en TD.

ORGANISATION DU TRAVAIL

Le projet est un travail par équipes. Voici les contraintes pour la constitution des équipes du projet :

- Maximum huit équipes dans un groupe de TD
- Minimum trois étudiants par équipe
- Maximum six étudiants par équipe
- Les délégués doivent envoyer un tableau excel avec la constitution des équipes au plus tard le dimanche 25 octobre à 23h59. A défaut d'une constitution des équipes fournie par les délégués de chaque groupe TD, les enseignants pourront décider eux-mêmes de la constitution des équipes.

Langage de programmation

- Au choix : C, C++, Python, Java
- Le langage choisi doit cependant être suffisamment maîtrisé par tous les membres d'une même équipe afin que tous puissent participer.

SOUTENANCE

Durée : 30 minutes par équipe.

- Il n'y aura pas d'ordinateur mis à votre disposition. Vous devez impérativement venir avec le vôtre en vous assurant préalablement que celui-ci marche et compile.
- 30 minutes par équipe. Vous aurez 10 minutes au début pour faire une démonstration de votre programme, puis 20 minutes de questions/réponses. Inutile donc de préparer un diaporama de présentation.
- La note de la soutenance est collective. Durant la soutenance, votre enseignant pourra poser n'importe quelle question à n'importe quel membre de l'équipe, qu'il participe à l'élaboration de la partie du programme concerné ou non. Tous les membres de l'équipe sont donc responsables de s'assurer que tous les autres membres ont le même degré de compréhension du projet.

Les soutenances ont une durée limitée. Votre enseignant a un planning très serré. Il ne pourra pas continuer les soutenances après le créneau horaire prévu. Il vous est donc vivement conseillé d'être vraiment prêt à l'heure du début de votre soutenance, ce qui implique :

- Attendre à la porte de la salle et entrer dès que c'est à votre tour ;
- Avoir préparé votre ordinateur, y avoir inclus tous vos programmes et fichiers contenant les tableaux de contraintes ;
- Avoir vérifié le chargement de la batterie de l'ordinateur ;
- L'avoir démarré et mis en mode veille ;
- Avoir un ordinateur de secours sur lequel vous avez aussi vérifié le bon fonctionnement de votre programme, au cas où…

Tous les ans, il y a plein d'étudiants qui pensent ne jamais avoir de problème. Tous les ans, il y en a qui en ont. Résultat : ils sont stressés et ont moins de temps pour leur soutenance. Dommage…

RENDU DU TRAVAIL

La date limite pour déposer le travail, sur Moodle, est le **dimanche 6 décembre à 23h59.**

Le contenu du rendu :

- Code source : Tout fichier code **que vous avez tapé vous-même** à l'exclusion de tout autre fichier (**donc aucun fichier produit par le logiciel durant la compilation ou exécution)**, bien commenté.
- Les traces d'exécution sous forme de fichiers .txt, un fichier par tableau de test. Vous devrez exécuter votre programme sur l'intégralité des tableaux de test et fournir les traces d'exécution correspondantes. Attention, les copies d'écran ne sont pas acceptées.
- Un tableau de contraintes non testé, ou pour lequel les traces d'exécution ne seront pas fournies, sera considéré comme un cas sur lequel votre programme ne fonctionne pas correctement.
- Tout fichier que vous utilisez (et mettez sur Moodle) doit être préfixé par votre numéro d'équipe : par exemple, si vous avez un fichier « main » et si vous utilisez le langage C++, et que vous êtes dans l'équipe B2, ce fichier doit avoir le nom B2-main.cpp (ou B2_main.cpp).

DEROULEMENT DU PROGRAMME

Mettre en place un programme qui exécute les actions suivantes préalables à l'ordonnancement :

1. Lecture d'un tableau de contraintes donné dans un fichier texte (.txt) et stockage en mémoire.
2. Affichage du graphe correspondant sous forme matricielle (matrice des valeurs). Attention : cet affichage doit se faire à partir du contenu mémoire, et non pas directement en lisant le fichier. Ce graphe doit incorporer les deux sommets fictifs α et ω (notés 0 et N+1 où N est le nombre de tâches).
3. Vérification des propriétés nécessaires du graphe pour qu'il puisse servir d'un graphe d'ordonnancement : pas de circuit, pas d'arcs à valeur négative.

   Si la réponse à la question 3 est « Oui », procéder au calcul des calendriers :

4. Calculer les rangs de tous les sommets du graphe.
5. Calculer le calendrier au plus tôt, le calendrier au plus tard et les marges.
6. Pour le calcul du calendrier au plus tard, considérez que la date au plus tard de fin de projet est égale à sa date au plus tôt.
7. Calculer le(s) chemin(s) critique(s) et les afficher.

Lors de son exécution, afin de faciliter le déroulement de votre soutenance, votre programme doit être capable de « boucler » sur une série de tableaux de contraintes que vous aurez préparées. Stopper votre programme après chaque graphe puis le relancer n'est pas une bonne solution, et cela entrainera une pénalité.

La structure globale de votre programme est illustrée par le pseudo-code suivant :

```
Début
    Tant que l'utilisateur décide de tester un tableau de
    contraintes faire
        Choisir le tableau de contraintes à traiter

        Lire le tableau de contraintes sur fichier et le
        stocker en mémoire

        Créer la matrice correspondant au graphe
        représentant ce tableau de contraintes et
        l'afficher

        Vérifier que les propriétés nécessaires pour que ce
        graphe soit un graphe d'ordonnancement sont
        vérifiées

            SI oui alors
                Calculer les rangs des sommets et les
                afficher
                Calculer les calendriers au plus tôt et
                au plus tard et les afficher
                Calculer les marges et les afficher
```

```
                        Calculer le(s) chemin(s) critique(s) et
                        les afficher
                    Sinon
                        Proposer à l'utilisateur de changer de
                        tableau de contraintes
                fin Tant que
        Fin
```

Remarque : Il est bien évident qu'il est possible de mettre en œuvre la détection de circuit dans l'algorithme de calcul de rang. Vous n'affichez les rangs que dans l'absence de circuit.

TABLEAUX DE CONTRAINTES A PRENDRE EN COMPTE

Un tableau de contraintes aura une forme suivante (on prend comme exemple le tableau C01 de la feuille de TD III, où l'on a remplacé les étiquettes des tâches alphabétiques par des nombres (A→1, B→2 etc.) et où sur chaque ligne le premier chiffre est le numéro de tâche, le deuxième sa durée, et les autres chiffres, si présents, sont des contraintes (prédécesseurs)) :

1 9
2 2
3 3 2
4 5 1
5 2 1 4
6 2 5
7 2 4
8 4 4 5
9 5 4
10 1 2 3
11 2 1 5 6 7 8

Les N tâches sont numérotées de 1 à N. La tâche fictive α sera notée 0. La tâche fictive ω aura le numéro N+1.

Votre programme doit être capable d'importer un tableau de contraintes quelconque répondant aux critères ci-dessus, y compris au cas où le graphe correspondant à ce tableau contiendra des circuits, et le transformer en un graphe sous forme matriciel (une matrice de valeurs).

TABLEAUX DE TEST

Pour vous permettre de tester votre programme, des tableaux de test vous seront fournis **le lundi 23 novembre** sous forme de fichiers .txt. Ces fichiers doivent être disponibles sur votre ordinateur au moment de la soutenance.

N'attendez pas l'arrivée des tableaux de contraintes de test pour tester votre programme ! Il serait alors trop tard. N'hésitez pas, pendant votre développement, à utiliser tous les tableaux de contraintes vus en cours, dans les feuilles d'exercices, et créer vos propres exemples. Plus vous ferez de tests, plus vous pourrez être certains que votre programme fonctionne correctement.

Attention : pendant la soutenance, votre programme sera également testé sur des nouveaux fichiers texte qui vous seront fournis sur place.

TRACES D'EXECUTION

Chacune des étapes doit afficher des traces du déroulement de l'algorithme. En voici un exemple qui correspond à un tableau de contraintes suivant :

1 1
2 2
3 3 1
4 4 1 2
5 5 2 4

Nous donnons un exemple d'affichage pour les trois premières étapes, les autres étapes doivent suivre le même principe de lisibilité.

Étape	Exemple de trace (ce ne sont que des exemples : vous pouvez faire ce que vous voulez, pourvu que l'on comprenne vite et sans problème comment votre algorithme fonctionne)

1	*Affichage du graphe comme un jeu de triplets , par exemple :*
	`* Création du graphe d'ordonnancement :`
	`7 sommets`
	`9 arcs`
	`0 -> 1 = 0`
	`0 -> 2 = 0`
	`1 -> 3 = 1`
	`1 -> 4 = 1`
	`2 -> 4 = 2`
	`2 -> 5 = 2`
	`3 -> 6 = 3`
	`4 -> 5 = 4`
	`5 -> 6 = 5`

2	`Matrice des valeurs`
	`  0 1 2 3 4 5 6`
	`0 * 0 0 * * * *`
	`1 * * * 1 1 * *`
	`2 * * * * 2 2 *`
	`3 * * * * * * 3`
	`4 * * * * * 4 *`
	`5 * * * * * * 5`
	`6 * * * * * * *`
	(Veillez à ce que la matrice soit affichée avec des colonnes bien alignées et avec les entêtes des lignes et colonnes.)

3	Il y a un seul point d'entrée, 0
	Il y a un seul point de sortie, 6
	Détection de circuit (*Par exemple, avec la méthode de suppression des points d'entrée) :*
	`* Détection de circuit`
	`* Méthode d'élimination des points d'entrée`
	`Points d'entrée : 0`
	`Suppression des points d'entrée`
	`Sommets restants : 1 2 3 4 5 6`
	`Points d'entrée : 1 2`
	`Suppression des points d'entrée`
	`Sommets restants : 3 4 5 6`
	`Points d'entrée : 3 4`
	`Suppression des points d'entrée`
	`Sommets restants : 5 6`
	`Points d'entrée : 5`
	`Suppression des points d'entrée`
	`Sommets restants : 6`
	`Points d'entrée : 6`
	`Suppression des points d'entrée`
	`Sommets restants : Aucun`
	`-> Il n'y a pas de circuit`
	Il n'y a pas d'arcs négatifs
	–> C'est un graphe d'ordonnancement